

Week 1- Fundamentals

Instructors- Dr. F Adamu-Fika

Ms. OV Okpoko

Learning Objectives

- Define algorithms and understand their importance in computing.
- Classify algorithms by function or problem type (e.g., sorting, searching).
- Explore classification by algorithmic paradigm (e.g., divide-and-conquer, greedy).
- Set the foundation for analysing algorithms in terms of efficiency and complexity.

Definition of an Algorithm

A step-by-step procedure or formula for solving a problem.

Characteristics of an Algorithm:

- Finiteness: Terminates after a finite number of steps.
- Definiteness: Clear and unambiguous.
- Input: Takes zero or more inputs.
- Output: Produces at least one output.
- Effectiveness: All steps are basic enough to be carried out.

Examples include sorting, searching, finding the shortest path algorithms

Importance and application of algorithms

- Core of computer science and software development.
- Enable efficient problem-solving.
- Apply to various domains (e.g., networks, AI, databases).

Real-World Applications of algorithms

- Navigation (e.g., GPS pathfinding).
- E-commerce (e.g., search and recommendations).
- Cybersecurity (e.g., encryption and hashing).

Classification by Function or Problem Type

Each type of algorithm solves a specific computational need and is chosen based on the problem requirements. They include

- Sorting Algorithms: Arrange data in a specific order (e.g., Quick Sort, Merge Sort).
- Searching Algorithms: Locate specific data within a dataset (e.g., Binary Search, Linear Search).
- Graph Algorithms: Solve problems related to graph structures (e.g., BFS, DFS, Dijkstra's algorithm). •
- Optimisation Algorithms: Find the best solution under given constraints (e.g., Knapsack problem).

- Numerical Algorithms: Perform calculations, such as root-finding or numerical integration (e.g., NewtonRaphson method).
- String Algorithms: Manipulate and process strings (e.g., pattern matching with KMP).
- Combinatorial Algorithms: Solve problems involving combinations, permutations, or subsets (e.g., Traveling Salesman Problem).
- Decision Problems: Provide a yes or no answer to a specific question (e.g., satisfiability problem).

Classification by Algorithmic Paradigm

Paradigms are high-level strategies for approaching a range of computational problems. The following are algorithmic paradigms:

- **Brute-Force:**
 - Approach that exhaustively searches all possible solutions to find an answer.
 - Example: Checking all possible subsets in the subset-sum problem.
- **Divide-and-Conquer:**
 - Approach that breaks down a problem into smaller subproblems, solves them independently, and combines results.
 - Example: Merge Sort, Quick Sort.

- Decrease-and-Conquer:
 - Approach that reduces a problem to a smaller instance and builds up a solution.
 - Types of Decrease-and-Conquer:
 - Decrease by a Constant (e.g., Insertion Sort).
 - Decrease by a Factor (e.g., Binary Search).
 - Decrease-and-Conquer with Variable Size Decrease (e.g., Euclidean algorithm for GCD).
- Dynamic Programming:
 - Approach that solves problems by breaking them into overlapping subproblems and storing results to avoid redundant calculations.
 - Examples: Fibonacci sequence, Knapsack problem.

- Transform-and-Conquer:
 - Approach that transforms the problem into another form that is easier to solve.
 - Types of Transform-and-Conquer:
 - Instance Simplification (e.g., Heapsort).
 - Representation Change (e.g., balanced search trees).
 - Problem Reduction (e.g., transforming shortest path problems in graphs).
- Greedy Algorithms:
 - Approach that makes the locally optimal choice at each step to find an overall optimal solution.
 - Examples:
 - Prim's Algorithm for MST.
 - Dijkstra's Algorithm for shortest paths in graphs with non-negative weights.
 - Huffman Coding for data compression.

- Backtracking:
 - Incrementally builds solutions and abandons a candidate as soon as it determines it cannot be valid.
 - Examples: N-Queens problem, Sudoku solver.
- Branch and Bound:
 - Solves optimisation problems by reducing the search space.
 - Example: Traveling salesman problem.
- Approximation Algorithms:
 - Finds an approximate solution when exact solutions are computationally infeasible.
 - Example: Approximation for NP-hard problems like the traveling salesman.

Structure of Algorithms

The structure an algorithm describes the organisation of an algorithm, not its function.

- Recursive Structure:
 - An algorithm that calls itself to solve smaller subproblems.
 - Example: Recursive factorial calculation.
- Iterative Structure:
 - Uses loops to repeat steps until a condition is met.
 - Example: Iterative factorial calculation.

Examples of Recursive structure

Algorithm POW2(n)

Input: n , a natural number

Output: n^{th} power of 2

START

1. If ($n = 0$)
2. return 1
3. else
4. return $2 * \text{POW2}(n - 1)$
5. END

Algorithm EVENSUM(n)

Input: n , a positive integer

Output: sum of the n^{th} even natural number (the first being 0)

START

1. if ($n==1$)
2. return 0
3. else
4. return EVENSUM($n - 1$) + ($2n - 2$)

END

Examples of Iterative structure

Algorithm POW(n)

Input: n , a natural number

Output: n^{th} power of 2

START

1. $i = 0$
2. $power = 1$
3. while ($i < n$)
4. $power = 2 * power$
5. $i = i + 1$
6. return power

END

Algorithm EVENSUM(n)

Input: n , a positive integer

Output: sum of the n^{th} even natural number (the first being 0)

START

1. $i = 1$
2. $even = 0$
3. while ($i \leq n$)
4. $even = even + (2i - 2)$
5. $i = i + 1$
6. return $even$

END

Pros and Cons

- Recursive algorithms are often more readable but may use more memory.
- Iterative algorithms are generally, more efficient in terms of space.

Efficiency and Complexity

- Efficiency: How quickly and effectively an algorithm solves a problem.
- Complexity: Understanding complexity helps conserve resources, especially for large datasets.
 - Time Complexity: Measures runtime as a function of input size.
 - Space Complexity: Measures memory usage.

Real-World Examples of Algorithms

- Sorting: Quick Sort for database indexing.
- Searching: Binary Search in search engines.
- Pathfinding: Dijkstra's Algorithm for GPS navigation.
- Cybersecurity: Encryption algorithms for secure data transmission.

Online resources

- MIT OpenCourseWare – “Introduction to Algorithms”. URL: MIT OpenCourseWare
This free MIT course includes lectures based on the “Introduction to Algorithms” textbook (CLRS). Recommended lectures introduce algorithms, basic paradigms, and complexity concepts.
- Khan Academy – “Algorithms”. URL: Khan Academy
Beginner-friendly tutorials on basic algorithm concepts, classifications, and introductory complexity analysis. Interactive exercises reinforce foundational knowledge.
- GeeksforGeeks – Basics of Algorithms. URL: GeeksforGeeks
GeeksforGeeks offers articles on introductory algorithms, including sorting, searching, and recursive and iterative structures. The resource includes sample code and explanations for each topic.
- VisuAlgo – URL: VisuAlgo
Visual representations of basic algorithms, helping students understand recursive and iterative structures through interactive animations. This is a useful resource for visual learners.
- Coursera – “Algorithms Specialization” by Stanford University. URL: Coursera
Week 1 includes fundamental concepts in algorithms and algorithmic thinking, taught by Professor Tim Roughgarden.